WIKIPEDIA 25
25 years of the free encyclopedia

Memory hierarchy

In computer architecture, the **memory hierarchy** separates computer storage into a hierarchy based on response time. Since response time, complexity, and capacity are related, the levels may also be distinguished by their performance and controlling technologies.[1] Memory hierarchy affects performance in computer architectural design, algorithm predictions, and lower level programming constructs involving locality of reference.

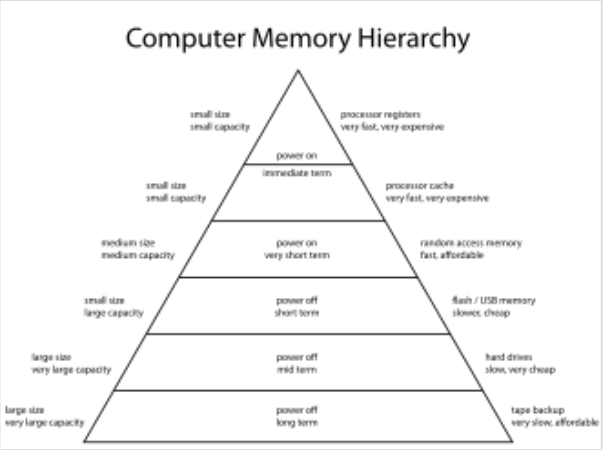


Diagram of the computer memory hierarchy

Designing for high performance requires considering the restrictions of the memory hierarchy, i.e. the size and capabilities of each component. Each of the various components can be viewed as part of a hierarchy of memories $(m_1, m_2, ..., m_n)$ in which each member m_i is typically smaller and faster than the next highest member m_{i+1} of the hierarchy. To limit waiting by higher levels, a lower level will respond by filling a buffer and then signaling for activating the transfer.

There are four major storage levels.[1]

- *Internal* – processor registers and cache.
- Main – the system RAM and controller cards.
- On-line mass storage – secondary storage.
- Off-line bulk storage – tertiary and off-line storage.

This is a general memory hierarchy structuring. Many other structures are useful. For example, a paging algorithm may be considered as a level for virtual memory when designing a computer architecture, and one can include a level of nearline storage between online and offline storage.

Properties of the technologies in the memory hierarchy

- Adding complexity slows the *memory hierarchy*.[2]
- CMOx memory technology stretches the flash space in the memory hierarchy[3]
- One of the main ways to increase system performance is minimising how far down the memory hierarchy one has to go to manipulate data.[4]
- Latency and bandwidth are two metrics associated with caches. Neither of them is uniform, but is specific to a particular component of the memory hierarchy.[5]
- Predicting where in the memory hierarchy the data resides is difficult.[5]
- The location in the memory hierarchy dictates the time required for the prefetch to occur.[5]

Examples

The number of levels in the memory hierarchy and the performance at each level has increased over time. The type of memory or storage components also change historically.[6]



Memory hierarchy of an AMD Bulldozer server as detected by hwloc's `lstopo` tool

Cache, memory, and external storage hierarchy of a 2020s computer system (AMD Zen 4)

Level		Size	Throughput	Latency	Notes
Register file		18,432 bits	Up to 256 GB/s (512 bits/cycle)	0.25 ns (1 cycle)[7]	All CPU-related conversion assumes a 4.0 GHz clock. Same for below. Full utilization of throughput is impossible on real workloads. Size is provided for each core.
CPU cache	L1 data	32 KiB	Up to 64 GB/s (64 bytes/4 cycles)	1 ns (4 cycles)[7]	Hardware prefetching is required for maximum throughput. Size and throughput are per-core. Code cache has the same size but is not manipulable as data.
	L2	1 MB	Up to 18.3 GB/s (64 bytes/14 cycles)	3.5 ns (14 cycles)[7]	Size and throughput are per-core.
	L3	16–32 MB	Up to 5.45 GB/s (64 bytes/47 cycles)	11.75 ns (47 cycles)[7]	Size is shared among 8 cores. Throughput is per-core.
Main memory (primary)		64 GiB	~60 GB/s	82.5 ns	Size is shared among all cores. Latency depends on the memory clock and memory timings. In this case, a result from a pair of 32 GB DDR5 DIMMs set to 6000 MT/s via the factory EXPO profile is used.[8] Systems with multiple CPU sockets have an additional NUMA delay when a CPU tries to access memory under the control of another NUMA node.
Mass storage (secondary)	Solid-state drive	2 TB	2000 MB/s	0.2 ms	Figures for a M.2 NVMe SSD from 2017, the Samsung 960 Pro.[9]
	Hard disk drive	18 TB	500 MB/s	4.16 ms	Per-drive figures for Exos 2X18 (ST18000NM0092), an enterprise-grade 3.5 inch SATA HDD.[10]
Nearline (tertiary)	Spun-down HDDs (MAID)	Petabytes		25 s	Per-drive figures for Exos 2X18 (ST18000NM0092), from user manual entry for "start/stop times".[11] In a typical MAID setup, hundreds of spun-down HDDs may be used for petabytes of storage.
	Tape library	Exabytes	160 MB/s[12]	Minutes	
Offline storage		Exabytes	Depends on medium	Depends on human operation	

Some CPUs include additional levels of cache between L3 and memory. For example, the Haswell microarchitecture includes an L4 cache of 128 MB on mobile units.[13][14]

The lower levels of the hierarchy – from mass storage downwards – are also known as tiered storage. The formal distinction between online, nearline, and offline storage is:[15]

- Online storage is immediately available for I/O.
- Nearline storage is not immediately available, but can be made online quickly without human intervention.
- Offline storage is not immediately available, and requires some human intervention to bring online.

For example, always-on spinning disks are online, while spinning disks that spin down, such as massive arrays of idle disk (MAID), are nearline. Removable media such as tape cartridges that can be automatically loaded, as in a tape library, are nearline, while cartridges that must be manually loaded are offline.

Programming

Most modern CPUs are so fast that, for most program workloads, the bottleneck is the locality of reference of memory accesses and the efficiency of the caching and memory transfer between different levels of the hierarchy. As a result, the CPU spends much of its time idling, waiting for memory I/O to complete. This is sometimes called the *space cost*, as a larger memory object is more likely to overflow a small and fast level and require use of a larger, slower level. The resulting load on memory use is known as *pressure* (respectively *register pressure*, *cache pressure*, and (main) *memory pressure*). Terms for data being missing from a higher level and needing to be fetched from a lower level are, respectively: register spilling (due to register pressure: register to cache), cache miss (cache to main memory), and (hard) page fault (*real* main memory to *virtual* memory, i.e. mass storage, commonly referred to as *disk* regardless of the actual mass storage technology used).

Modern programming languages mainly assume two levels of memory, main (*working*) memory and mass storage. The exception is the relatively low-level assembly language and in the inline assemblers of higher-level languages such as C where "prefetch" instructions can be used to preload the cache. Taking optimal advantage of the memory hierarchy requires the cooperation of programmers, hardware, and compilers (as well as underlying support from the operating system):

- *Programmers* are responsible for moving data between disk and memory through file I/O.
- *Hardware* is responsible for moving data between memory and caches.
- *Optimizing compilers* are responsible for generating code that, when executed, will cause the hardware to use caches and registers efficiently.

Many programmers assume one level of memory. This works fine until the application hits a performance wall. At that point, the programmer needs to change the code's memory access patterns to that it works well with cache resources. A classic illustration of the effect of locality and caching is in the form of changing the order of iterating a three-dimensional array. *Computer Systems: A Programmer's Perspective* is a classic textbook that deals with this aspect of systems programming.[16]

Memory tiering

Memory tiering is the practice of dividing the main memory into several levels by their performance characteristics, and (in analogy to storage tiering) moving memory content between them. What appears to be the "main memory" (physical address space) can be made up of heterogeneous parts due to NUMA, CXL-attached memory (on PCIe slots), Optane DCPMMs, or memory found on other expansion hardware such as coprocessors and GPUs, hence the need for tiering. Swapping can also be considered a form of tiering.[17]

Memory tiering is implemented on Linux as an extension to NUMA, where each memory provider has a CPU-less NUMA node with an appropriate "abstract distance" reflecting its performance. The existing scheme for migrating memory between NUMA nodes using "hotness" indicated by page faults is adapted to tiering by Huang Ying (Al Maruf's TPP scheme is not in Linux mainline).[18] It also uses a weighted-interleave allocation policy.[19]

See also

- Cache hierarchy
- Use of spatial and temporal locality: hierarchical memory
- Buffer vs. cache
- Cache hierarchy in a modern processor
- Memory wall
- Computer memory
- Hierarchical storage management
- Cloud storage
- Memory access pattern
- Communication-avoiding algorithm

References

1. Toy, Wing; Zee, Benjamin (1986). *Computer Hardware/Software Architecture* (https://archive.org/details/computerhardware0000toyw/page/30). Prentice Hall. p. 30 (https://archive.org/details/computerhardware0000toyw/page/30). ISBN 0-13-163502-6.
2. Write-combining
3. "Memory Hierarchy" (https://web.archive.org/web/20090805170055/http://www.unitysemi.com/applications-memory-hierarchy.html). Unitity Semiconductor Corporation. Archived from the original (http://www.unitysemi.com/applications-memory-hierarchy.html) on 5 August 2009. Retrieved 16 September 2009.
4. Pádraig Brady. "Multi-Core" (http://www.pixelbeat.org/docs/memory_hierarchy/). Retrieved 16 September 2009.
5. van der Pas, Ruud (2002). "Memory Hierarchy in Cache-Based Systems" (http://www.sun.com/blueprints/1102/817-0742.pdf) (PDF). Santa Clara, California: Sun Microsystems: 26. 817-0742-10. `{{cite journal}}`: Cite journal requires `|journal=` (help)
6. "Memory & Storage – Timeline of Computer History – Computer History Museum" (http://www.computerhistory.org/timeline/memory-storage/). *www.computerhistory.org*.
7. Fog, Agner. "The microarchitecture of Intel and AMD CPUs" (https://www.agner.org/optimize/microarchitecture.pdf) (PDF). Chapters used: 24.16 Cache and memory access (Zen 4).

8. "AMD Ryzen 7000/9000 DDR5 RAM OC Guide XPM and EXPO Profile Benchmarks" (https://www.ocinside.de/workshop_en/amd_ryzen_7000_9000_ddr5_oc_guide/3/).
9. "Samsung 960 Pro M.2 NVMe SSD Review" (http://www.storagereview.com/samsung_960_pro_m2_nvme_ssd_review). storagereview.com. 20 October 2016. Retrieved 2017-04-13.
10. "Datasheet Exos 2X18" (https://www.seagate.com/content/dam/seagate/migrated-assets/www-content/datasheets/pdfs/exos-2x18-DS2093-1-2202GB-en_SG.pdf) (PDF).
11. "2X18 SATA Product Manual" (https://www.seagate.com/content/dam/seagate/migrated-assets/www-content/manuals/exos-x-2x18/pdf/203859600a.pdf) (PDF).
12. "Ultrium – LTO Technology – Ultrium GenerationsLTO" (https://web.archive.org/web/20110727052050/http://www.lto.org/technology/generations.html). Lto.org. Archived from the original (http://www.lto.org/technology/generations.html) on 2011-07-27. Retrieved 2014-07-31.
13. Crothers, Brooke. "Dissecting Intel's top graphics in Apple's 15-inch MacBook Pro – CNET" (https://news.cnet.com/8301-13579_3-57609045-37/dissecting-intels-top-graphics-in-apples-15-inch-macbook-pro/). News.cnet.com. Retrieved 2014-07-31.
14. "SiSoftware Zone" (https://web.archive.org/web/20140913231938/http://www.sisoftware.co.uk/?d=qa&f=mem_hsw). Sisoftware.co.uk. Archived from the original (http://www.sisoftware.co.uk/?d=qa&f=mem_hsw) on 2014-09-13. Retrieved 2014-07-31.
15. Pearson, Tony (2010). "Correct use of the term Nearline" (https://web.archive.org/web/20181127020712/https://www.ibm.com/developerworks/community/blogs/InsideSystemStorage/entry/the_correct_use_of_the_term_nearline2?lang=en). *IBM Developerworks, Inside System Storage*. Archived from the original (https://www.ibm.com/developerworks/community/blogs/InsideSystemStorage/entry/the_correct_use_of_the_term_nearline2) on 2018-11-27. Retrieved 2015-08-16.
16. "A Programmer's Perspective: Memory Systems" (https://csapp.cs.cmu.edu/3e/perspective.html).
17. Corbet, Jonathan (May 12, 2023). "The future of memory tiering" (https://lwn.net/Articles/931421/). *LWN*.
18. "NUMA balancing: optimize page placement for memory tiering system · torvalds/linux@c574bbe" (https://github.com/torvalds/linux/commit/c574bbe917036c8968b984c82c7b13194fe5ce98). *GitHub*.
19. Corbet, Jonathan (October 25, 2023). "Weighted interleaving for memory tiering" (https://lwn.net/Articles/948037/). *LWN*.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Memory_hierarchy&oldid=1342917287"